

Quality Assurance (QA)

Boas vindas!

Olá! Se você chegou até aqui, seu olhar crítico e busca pela excelência te trouxeram ao lugar certo. Ser um Analista de QA é ser o guardião da experiência do usuário e da estabilidade do sistema.

Estamos aqui para ajudar em sua jornada, então não hesite em nos procurar caso surjam dúvidas. Boa sorte nesta nova etapa!

Instruções

Você tem **total liberdade** para escolher as ferramentas que desejar para realizar os testes e a documentação (ex: Cypress, Postman, Insomnia, etc.).

→ O desafio consiste em realizar uma auditoria completa de qualidade no projeto **NaSalinha**. Você deve identificar bugs (propositais ou não) e documentá-los.

- **Link do Repositório:** [Repositório](#)

Requisitos

1. O Início: Seu Repositório

Crie um repositório no GitHub (seu "Portfólio de Qualidade") contendo:

- **README.md:** Descrição, tecnologias/ferramentas e tipos de testes.
- **Coleção de API:** Arquivo JSON exportado do seu teste no **Insomnia** ou **Postman**.
- **Documentação (Escolha UMA das opções abaixo):**

- **OPÇÃO A (Integrada):** GitHub Issues para os Bug Reports e GitHub Projects para o Kanban e Casos de Teste.
- **OPÇÃO B (Documental):** Arquivos separados (PDF, Markdown ou Planilha) dentro de uma pasta [/docs](#).

2. O Que Testar (As 3 Funcionalidades "Core")

Acesse o documento de requisitos de cada uma das 3 áreas abaixo:

1. **Autenticação JWT:** Autenticação e Gestão de Acesso.
2. **Check-in por Foto:** Upload de provas e validação de mídia.
3. **Sistema de Pontos:** Cálculo correto, persistência no banco e reflexo no ranking.

Acesse o documento de requisitos dessas 3 áreas : [Requisitos](#)

3. Casos de Teste (O Planejamento)

Você deve elaborar uma estrutura de testes para **cada uma** das 3 áreas Core. O planejamento deve conter:

- **Quantidade:** 2 casos de teste por requisito.
- **Distribuição por Tipo:** Para cada uma das 3 áreas (JWT, Check-in e Pontos), você deve realizar:
 - a. **Caso Funcional:** Foco na interface (1 Positivo ou 1 Negativo).
 - b. **Caso de API:** Foco no servidor/Insomnia (1 Positivo ou 1 Negativo).
 - c. **Caso de Regressão:** Para o teste de regressão, escolha o fluxo principal da funcionalidade. **Simule** que um bug reportado por você foi corrigido e descreva quais testes você repetiria para garantir que essa correção não gerou efeitos colaterais em outras partes do sistema.

4. Bug Reports (A Execução)

Identificou uma falha durante os testes? Ela deve ser documentada. Encontre e documente pelo menos 2 Bug Reports por área , a qualidade do reporte é fundamental. **Existe um exemplo de template de bug report da página Artefatos na trilha de QA. Não é obrigatório segui-lo.**

5. Relatório de Testes (A Visão Geral)

Após concluir a execução de todos os seus casos de teste, você deve elaborar um **Relatório de Testes Final**. Este documento serve para comunicar o estado atual da qualidade do projeto e deve conter:

- **Métricas de Execução:** * Total de Casos de Teste planejados vs. executados.
 - Quantidade de testes que passaram (Pass) vs. falharam (Fail).
- **Distribuição de Severidade:** Quantos dos bugs encontrados são **Críticos** (travam o sistema), **Maiores** (causam erro de lógica) ou **Menores** (erros visuais).

6. Tipos de Testes Obrigatórios

- **Foco Funcional:** Validar se o comportamento na tela bate com o que é esperado.
- **Validação de API:** Testar endpoints e Status Codes.
- **Teste de Regressão:** Garantir que novas correções não "quebraram" o que já estava funcionando.

Ciclo de Vida do Teste: Como as Ferramentas se Integram

Para garantir uma auditoria de qualidade, siga este fluxo de trabalho:

1. **Planeje:** Antes de abrir o sistema, escreva seus Casos de Teste obrigatórios (conforme as 3 áreas Core). Defina o que você vai testar e qual o resultado esperado.
2. **Execute:** Utilize o navegador para os testes de interface e o Insomnia/Postman para validar os endpoints da API. Acompanhe os logs no terminal do Docker para identificar falhas invisíveis na tela.
3. **Documente:** Se o teste falhar: Abra imediatamente um Bug Report detalhado, descrevendo a falha técnica encontrada.
4. **Relatório de Testes :** Ao final, adicione os resultados ao relatório de testes.

O que fazer com o "Bug Extra"?

Se durante os seus testes você encontrar um erro que **não estava** nos seus casos de teste iniciais, você deve documentar! Mas não basta só abrir o Bug Report. Para manter a **rastreabilidade** (que é fundamental para um QA), siga estes passos:

1. **Registre o Caso de Teste:** Adicione esse novo cenário à sua lista como um **Caso de Teste Ad-hoc** (Extra). Isso mostra que você teve olhar crítico para ir além do que planejou no papel.
2. **Abra o Bug Report:** Use o template oficial para detalhar a falha técnica, os logs e as evidências.
3. **Evidências de testes:** Adicione ao bug report uma evidência do erro.

Cronograma de QA (7 Semanas)

Semanas 1-2: Setup, Exploração e Documentação

- **Foco:** Infraestrutura e entendimento do produto.
- **Ações:**
 - Subir o ambiente local usando **Docker**.
 - Criar o repositório no **GitHub** e estruturar as pastas.
 - Navegar pelo sistema para entender o fluxo do usuário.
 - **README:** Redigir a documentação inicial (tecnologias, como rodar o projeto e estratégia de testes).

Semana 3: Planejamento e Design de Testes

- **Foco:** O que testar?
- **Ações:**
 - Definir os cenários para as **3 Áreas Core** do sistema.
 - Escrita detalhada dos **Casos de Teste** (positivos e negativos)
 - Configuração da ferramenta de gestão (Jira, Trello ou GitHub Projects).

Semana 4: Execução de Testes Manuais e de Interface

- **Foco:** Quebrar o sistema via Front-end.
- **Ações:**
 - Execução dos casos de teste planejados na semana anterior.
 - Registro dos bugs encontrados.

Semana 5: Testes de API e Integração

- **Foco:** Validar as "entranhas" do sistema.
- **Ações:**
 - Utilizar ferramentas como **Postman** ou **Insomnia**.
 - Validar status codes (200, 201, 400, 404, 500).
 - Verificar se as regras de negócio do backend estão funcionando corretamente.
 - Registro dos bugs encontrados.

Semana 6: Testes de Regressão e Re-teste

- **Foco:** Garantir estabilidade.
- **Ações:**
 - Validar se os bugs reportados anteriormente foram corrigidos.
 - **Teste de Regressão:** Descrever uma situação simulada de teste de regressão.
 - Refinar os relatórios de bugs.

Semana 7: Finalização, Revisão e Entrega

- **Foco:** Qualidade da entrega final.
- **Ações:**
 - Revisão final de toda a documentação (README, Casos de Teste e Bug Reports).
 - Gravação do **vídeo de demonstração** (máximo 10 min), focando em mostrar o ambiente rodando e os bugs mais críticos encontrados.
 - Subir tudo para o GitHub e realizar o envio final.

Entrega

- Você deve gravar um vídeo de até 10 minutos demonstrando o funcionamento e explicando resumidamente a sua trajetória no projeto, como, por exemplo:

- Demonstração dos principais bugs encontrados e seus casos de testes.
- Exibição dos testes de API rodando no Insomnia/Postman
- Problemas enfrentados
- Como resolveu os problemas
- **Link do Repositório:** O link do repositório deve ser entregue ao orientador da trilha.

Ir Além (Diferenciais)

Se você finalizou os testes obrigatórios nas 3 funcionalidades "Core" (JWT, Check-in e Pontos) e quer se destacar na sua avaliação final, explore os seguintes pontos:

1. **Auditoria Completa (Todas as Funcionalidades):** Realizar o ciclo de testes também para as áreas de **Temporadas(CRUD completo)**.
2. **Kanban Organizado:** Demonstrar uma gestão da evolução dos seus testes através das colunas: *Ready for Test, In test, Test done*.
3. **Cronograma:** Fazer as atividades nos prazos do cronograma ou antes.